

Lab 3: The Geometric Solver

EE0849 Introduction to Robotics

Basri Erdogan

Table of contents

1	Lab Overview	2
1.1	Objectives	2
1.2	Prerequisites	2
1.3	Required Materials	2
1.4	Deliverables	2
2	Part 1: Understanding the 2-Link IK Problem (15 minutes)	3
2.1	1.1 Problem Statement	3
2.2	1.2 Setup	3
2.3	1.3 Exercise 1: Workspace Calculation	3
3	Part 2: Implementing Reachability Check (10 minutes)	4
3.1	2.1 Reachability Function	4
3.2	2.2 Exercise 2: Test Reachability	4
4	Part 3: Implementing the IK Solver (30 minutes)	5
4.1	3.1 The Complete IK Algorithm	5
4.2	3.2 Forward Kinematics for Verification	5
4.3	3.3 Exercise 3: Verify the IK Solution	5
5	Part 4: Visualizing Both Solutions (20 minutes)	7
5.1	4.1 Plot Both Configurations	7
5.2	4.2 Exercise 4: Multiple Target Points	8
6	Part 5: Edge Cases and Singularities (20 minutes)	9
6.1	5.1 Boundary Cases	9
6.2	5.2 Exercise 5: Understanding Singularities	9
6.3	5.3 Numerical Stability	9
7	Part 6: Solution Selection (15 minutes)	11
7.1	6.1 Minimum Motion Selection	11
7.2	6.2 Exercise 6: Path Following	11
8	Part 7: Animation (15 minutes)	13
8.1	7.1 Animate the Path	13
8.2	7.2 Exercise 7: Custom Path	14
9	Part 8: Using roboticstoolbox (Optional)	15
9.1	8.1 Compare with Library Implementation	15
9.2	8.2 Explore Different Initial Guesses	15

10 Summary and Cleanup	16
10.1 Checklist	16
10.2 Save Your Work	16
10.3 Key Takeaways	16
10.4 Next Week Preview	16

1 Lab Overview

1.1 Objectives

In this lab, you will:

1. Implement analytical inverse kinematics for a 2-link planar arm
2. Understand and visualize workspace boundaries
3. Handle both elbow-up and elbow-down solutions
4. Verify IK solutions using forward kinematics
5. Explore singularities and edge cases
6. Apply IK to path following

1.2 Prerequisites

- Completed Week 3 lab (forward kinematics)
- Understanding of trigonometry and `atan2` function
- Familiarity with numpy and matplotlib

1.3 Required Materials

- Computer with Python environment
- Lab notebook for calculations and sketches

1.4 Deliverables

By the end of this lab, you should have:

- Working `ik_2link()` function with both elbow configurations
- Workspace visualization with reachability regions
- Verification plots showing $FK(IK(target)) = target$
- Animation of the arm tracing a path

2 Part 1: Understanding the 2-Link IK Problem (15 minutes)

2.1 1.1 Problem Statement

Given a target position (x, y) and link lengths L_1, L_2 , find joint angles θ_1, θ_2 such that the end-effector reaches the target.

2.2 1.2 Setup

Create a new Python file `lab3_ik.py` with the following setup:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Wedge
from matplotlib.animation import FuncAnimation

# Link parameters (same as Lab 2)
L1 = 1.0 # meters
L2 = 0.5 # meters

# Plotting helper
def plot_arm(ax, theta1, theta2, L1, L2, color='steelblue', label=None):
    """Plot a 2-link arm at given configuration."""
    x0, y0 = 0, 0
    x1 = L1 * np.cos(theta1)
    y1 = L1 * np.sin(theta1)
    x2 = x1 + L2 * np.cos(theta1 + theta2)
    y2 = y1 + L2 * np.sin(theta1 + theta2)

    # Draw links
    ax.plot([x0, x1], [y0, y1], 'o-', color=color, linewidth=4,
            markersize=10, markerfacecolor='white', markeredgewidth=2)
    ax.plot([x1, x2], [y1, y2], 'o-', color=color, linewidth=4,
            markersize=10, markerfacecolor='white', markeredgewidth=2)

    # End effector
    ax.plot(x2, y2, 's', color=color, markersize=12, label=label)

    return x2, y2
```

2.3 1.3 Exercise 1: Workspace Calculation

Calculate the workspace boundaries for the arm:

Questions:

1. What is the maximum reach r_{max} ?
2. What is the minimum reach r_{min} ?
3. What is the area of the reachable workspace (annulus)?

Your Calculations:

- $r_{\max} = L_1 + L_2 = \$$ _____
- $r_{\min} = |L_1 - L_2| = \$$ _____
- Area $= (r_{\max}^2 - r_{\min}^2) = \$$ _____

3 Part 2: Implementing Reachability Check (10 minutes)

3.1 2.1 Reachability Function

Implement the reachability check:

```
def is_reachable(x, y, L1, L2):
    """
    Check if point (x, y) is reachable by the 2-link arm.

    Returns:
        True if reachable, False otherwise
    """
    r = np.sqrt(x**2 + y**2)
    r_min = abs(L1 - L2)
    r_max = L1 + L2

    return r_min <= r <= r_max
```

3.2 2.2 Exercise 2: Test Reachability

Test your function with these points:

```
test_points = [
    (1.0, 0.5), # Point A
    (1.5, 0.0), # Point B (on boundary)
    (0.0, 0.0), # Point C (origin)
    (2.0, 0.0), # Point D (too far)
    (0.3, 0.3), # Point E (close to inner boundary)
]

print("Reachability Test Results:")
print("-" * 40)
for x, y in test_points:
    r = np.sqrt(x**2 + y**2)
    reachable = is_reachable(x, y, L1, L2)
    status = "REACHABLE" if reachable else "UNREACHABLE"
    print(f"({x:4.1f}, {y:4.1f}): r = {r:.3f} m -> {status}")
```

Record your results:

Point	x	y	r	Reachable?
A	1.0	0.5		
B	1.5	0.0		
C	0.0	0.0		
D	2.0	0.0		
E	0.3	0.3		

4 Part 3: Implementing the IK Solver (30 minutes)

4.1 3.1 The Complete IK Algorithm

Implement the full IK function:

```
def ik_2link(x, y, L1, L2, elbow='up'):  
    """  
    Inverse kinematics for 2-link planar arm.  
  
    Parameters:  
        x, y: Target position (meters)  
        L1, L2: Link lengths (meters)  
        elbow: 'up' or 'down' configuration  
  
    Returns:  
        theta1, theta2: Joint angles (radians)  
  
    Raises:  
        ValueError: If point is unreachable  
    """  
    # Step 1: Check reachability  
    c2 = (x**2 + y**2 - L1**2 - L2**2) / (2 * L1 * L2)  
  
    if abs(c2) > 1.0:  
        raise ValueError(f"Point ({x}, {y}) is unreachable. |cos( )| = {abs(c2):.3f} > 1")  
  
    # Step 2: Calculate theta2  
    if elbow == 'up':  
        s2 = np.sqrt(1 - c2**2)  
    else: # elbow down  
        s2 = -np.sqrt(1 - c2**2)  
  
    theta2 = np.arctan2(s2, c2)  
  
    # Step 3: Calculate theta1  
    alpha = np.arctan2(L2 * s2, L1 + L2 * c2)  
    theta1 = np.arctan2(y, x) - alpha  
  
    return theta1, theta2
```

4.2 3.2 Forward Kinematics for Verification

Add the FK function to verify your IK:

```
def fk_2link(theta1, theta2, L1, L2):  
    """Forward kinematics for 2-link arm."""  
    x = L1 * np.cos(theta1) + L2 * np.cos(theta1 + theta2)  
    y = L1 * np.sin(theta1) + L2 * np.sin(theta1 + theta2)  
    return x, y
```

4.3 3.3 Exercise 3: Verify the IK Solution

Test the IK solver and verify with FK:

```

# Test point
x_target, y_target = 1.0, 0.5

print(f"Target: ({x_target}, {y_target})")
print("=" * 50)

# Solve for both configurations
for elbow in ['up', 'down']:
    theta1, theta2 = ik_2link(x_target, y_target, L1, L2, elbow)

    # Verify with FK
    x_check, y_check = fk_2link(theta1, theta2, L1, L2)

    print(f"\n{elbow.upper()} configuration:")
    print(f"    = {np.degrees(theta1):7.2f}°")
    print(f"    = {np.degrees(theta2):7.2f}°")
    print(f"    FK verification: ({x_check:.6f}, {y_check:.6f})")
    print(f"    Error: {np.sqrt((x_check-x_target)**2 + (y_check-y_target)**2):.2e} m")

```

Record your results:

Configuration	θ_1 (°)	θ_2 (°)	FK x	FK y	Error
Elbow Up					
Elbow Down					

5 Part 4: Visualizing Both Solutions (20 minutes)

5.1 4.1 Plot Both Configurations

Create a visualization showing both IK solutions:

```
def visualize_ik_solutions(x_target, y_target, L1, L2):
    """Visualize both elbow configurations for a target."""
    fig, ax = plt.subplots(figsize=(10, 8))

    # Draw workspace
    theta_range = np.linspace(0, 2*np.pi, 100)
    r_max = L1 + L2
    r_min = abs(L1 - L2)

    x_outer = r_max * np.cos(theta_range)
    y_outer = r_max * np.sin(theta_range)
    x_inner = r_min * np.cos(theta_range)
    y_inner = r_min * np.sin(theta_range)

    ax.fill(x_outer, y_outer, alpha=0.2, color='green')
    ax.fill(x_inner, y_inner, color='white')
    ax.plot(x_outer, y_outer, 'g--', linewidth=1, alpha=0.5)
    ax.plot(x_inner, y_inner, 'r--', linewidth=1, alpha=0.5)

    # Plot target
    ax.plot(x_target, y_target, 'k*', markersize=20,
           label=f'Target ({x_target}, {y_target})')

    # Plot both solutions
    colors = {'up': '#2196F3', 'down': '#FF5722'}

    for elbow in ['up', 'down']:
        try:
            theta1, theta2 = ik_2link(x_target, y_target, L1, L2, elbow)
            label = f'Elbow {elbow}: {np.degrees(theta1):.1f}°, {np.degrees(theta2):.1f}°'
            plot_arm(ax, theta1, theta2, L1, L2, color=colors[elbow], label=label)
        except ValueError as e:
            print(f"Elbow {elbow}: {e}")

    # Styling
    ax.set_xlim(-2, 2)
    ax.set_ylim(-1.5, 1.5)
    ax.set_aspect('equal')
    ax.grid(True, alpha=0.3)
    ax.set_xlabel('x (m)')
    ax.set_ylabel('y (m)')
    ax.set_title('2-Link IK: Elbow Up vs Elbow Down')
    ax.legend(loc='upper left')
    ax.plot(0, 0, 'ko', markersize=15) # Base

    plt.tight_layout()
    return fig, ax

# Run visualization
```

```
fig, ax = visualize_ik_solutions(1.0, 0.5, L1, L2)
plt.savefig('ik_both_solutions.png', dpi=150)
plt.show()
```

5.2 4.2 Exercise 4: Multiple Target Points

Test with different target points and observe the solutions:

```
targets = [
    (0.8, 0.8), # Upper region
    (1.2, 0.0), # On x-axis
    (0.0, 1.0), # On y-axis
    (-0.5, 0.5), # Second quadrant
]

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

for ax, (x, y) in zip(axes.flatten(), targets):
    # Your visualization code here for each target
    pass

plt.tight_layout()
plt.savefig('ik_multiple_targets.png', dpi=150)
plt.show()
```

Complete this exercise by adapting `visualize_ik_solutions()` for the subplot format.

6 Part 5: Edge Cases and Singularities (20 minutes)

6.1 5.1 Boundary Cases

Test what happens at workspace boundaries:

```
def test_boundary_cases(L1, L2):
    """Test IK at workspace boundaries."""
    r_max = L1 + L2
    r_min = abs(L1 - L2)

    test_cases = [
        (r_max, 0, "Maximum reach (fully extended)"),
        (r_min, 0, "Minimum reach (fully folded)"),
        (r_max * 0.99, 0, "Near maximum reach"),
        (r_min * 1.01, 0, "Near minimum reach"),
    ]

    print("Boundary Case Analysis")
    print("=" * 60)

    for x, y, description in test_cases:
        print(f"\n{description}: ({x:.4f}, {y:.4f})")
        try:
            for elbow in ['up', 'down']:
                theta1, theta2 = ik_2link(x, y, L1, L2, elbow)
                print(f"  {elbow:5s}:  ={np.degrees(theta1):7.2f}°,  ={np.degrees(theta2):7.2f}°")
        except ValueError as e:
            print(f"    Error: {e}")

test_boundary_cases(L1, L2)
```

6.2 5.2 Exercise 5: Understanding Singularities

Questions:

1. At maximum reach ($r = L_1 + L_2$), what is θ_2 ? Why?

Answer: _____

2. At minimum reach ($r = |L_1 - L_2|$), what is θ_2 ? Why?

Answer: _____

3. What happens to the two solutions (elbow up/down) at singularities?

Answer: _____

6.3 5.3 Numerical Stability

For points very close to boundaries, numerical errors can cause issues:

```
def ik_2link_safe(x, y, L1, L2, elbow='up', tol=1e-10):
    """IK with numerical tolerance for boundary cases."""
    c2 = (x**2 + y**2 - L1**2 - L2**2) / (2 * L1 * L2)

    # Clamp c2 to [-1, 1] with tolerance
    if c2 > 1.0 and c2 < 1.0 + tol:
```

```

    c2 = 1.0
elif c2 < -1.0 and c2 > -1.0 - tol:
    c2 = -1.0
elif abs(c2) > 1.0:
    raise ValueError(f"Point unreachable: |cos( )| = {abs(c2):.6f}")

# Rest of implementation same as before
if elbow == 'up':
    s2 = np.sqrt(1 - c2**2)
else:
    s2 = -np.sqrt(1 - c2**2)

theta2 = np.arctan2(s2, c2)
alpha = np.arctan2(L2 * s2, L1 + L2 * c2)
theta1 = np.arctan2(y, x) - alpha

return theta1, theta2

```

7 Part 6: Solution Selection (15 minutes)

7.1 6.1 Minimum Motion Selection

Implement a function that chooses the solution closest to the current configuration:

```
def ik_nearest(x, y, L1, L2, q_current):
    """
    Return IK solution closest to current configuration.

    Parameters:
        x, y: Target position
        L1, L2: Link lengths
        q_current: Current joint angles [theta1, theta2]

    Returns:
        theta1, theta2: Optimal joint angles
        config: 'up' or 'down'
    """
    solutions = []

    for elbow in ['up', 'down']:
        try:
            theta1, theta2 = ik_2link_safe(x, y, L1, L2, elbow)
            # Calculate "distance" in joint space
            dist = np.sqrt((theta1 - q_current[0])**2 +
                           (theta2 - q_current[1])**2)
            solutions.append((theta1, theta2, elbow, dist))
        except ValueError:
            pass

    if not solutions:
        raise ValueError("No valid IK solution found")

    # Return solution with minimum joint motion
    solutions.sort(key=lambda x: x[3])
    return solutions[0][0], solutions[0][1], solutions[0][2]
```

7.2 6.2 Exercise 6: Path Following

Use IK to make the arm trace a circle:

```
def trace_circle(cx, cy, radius, L1, L2, n_points=50):
    """Generate joint trajectory for circular path."""
    angles = np.linspace(0, 2*np.pi, n_points)

    trajectory = []
    q_current = [0.5, 0.5] # Initial guess

    for angle in angles:
        x = cx + radius * np.cos(angle)
        y = cy + radius * np.sin(angle)

        try:
            theta1, theta2, _ = ik_nearest(x, y, L1, L2, q_current)
```

```

        trajectory.append((theta1, theta2))
        q_current = [theta1, theta2]
    except ValueError:
        print(f"Point ({x:.2f}, {y:.2f}) unreachable")
        trajectory.append(None)

    return trajectory

# Trace a small circle centered at (0.8, 0.3)
trajectory = trace_circle(0.8, 0.3, 0.2, L1, L2)
print(f"Generated {len([t for t in trajectory if t])} valid waypoints")

```

8 Part 7: Animation (15 minutes)

8.1 7.1 Animate the Path

Create an animation of the arm following the circular path:

```
def animate_path(trajecory, L1, L2, save_path='arm_animation.gif'):
    """Animate the arm following a trajectory."""
    fig, ax = plt.subplots(figsize=(8, 8))

    # Setup
    ax.set_xlim(-2, 2)
    ax.set_ylim(-2, 2)
    ax.set_aspect('equal')
    ax.grid(True, alpha=0.3)

    # Draw workspace
    theta_range = np.linspace(0, 2*np.pi, 100)
    r_max = L1 + L2
    r_min = abs(L1 - L2)
    ax.plot(r_max * np.cos(theta_range), r_max * np.sin(theta_range),
            'g--', alpha=0.3)
    ax.plot(r_min * np.cos(theta_range), r_min * np.sin(theta_range),
            'r--', alpha=0.3)

    # Animation elements
    link1, = ax.plot([], [], 'b-', linewidth=4)
    link2, = ax.plot([], [], 'g-', linewidth=4)
    joints, = ax.plot([], [], 'ko', markersize=10)
    ee_marker, = ax.plot([], [], 'rs', markersize=12)
    ee_path, = ax.plot([], [], 'r-', linewidth=1, alpha=0.5)

    ee_history = []

    def init():
        link1.set_data([], [])
        link2.set_data([], [])
        joints.set_data([], [])
        ee_marker.set_data([], [])
        ee_path.set_data([], [])
        return link1, link2, joints, ee_marker, ee_path

    def update(frame):
        if trajectory[frame] is None:
            return link1, link2, joints, ee_marker, ee_path

        theta1, theta2 = trajectory[frame]

        # Joint positions
        x1 = L1 * np.cos(theta1)
        y1 = L1 * np.sin(theta1)
        x2 = x1 + L2 * np.cos(theta1 + theta2)
        y2 = y1 + L2 * np.sin(theta1 + theta2)

        link1.set_data([0, x1], [0, y1])
```

```

link2.set_data([x1, x2], [y1, y2])
joints.set_data([0, x1, x2], [0, y1, y2])
ee_marker.set_data([x2], [y2])

ee_history.append((x2, y2))
ee_path.set_data(*zip(*ee_history))

return link1, link2, joints, ee_marker, ee_path

ani = FuncAnimation(fig, update, frames=len(trajjectory),
                    init_func=init, blit=True, interval=50)

ani.save(save_path, writer='pillow', fps=20)
plt.close()
print(f"Animation saved to {save_path}")

# Create and save animation
animate_path(trajjectory, L1, L2)

```

8.2 7.2 Exercise 7: Custom Path

Design your own path (e.g., square, figure-8, line) and animate the arm following it.

9 Part 8: Using roboticstoolbox (Optional)

9.1 8.1 Compare with Library Implementation

```
from roboticstoolbox import DHRobot, RevoluteDH
from spatialmath import SE3

# Create 2-link robot
robot = DHRobot([
    RevoluteDH(d=0, a=L1, alpha=0),
    RevoluteDH(d=0, a=L2, alpha=0)
], name='2-Link')

# Target pose
T_target = SE3.Trans(1.0, 0.5, 0)

# Solve IK numerically
sol = robot.ikine_LM(T_target)

print("roboticstoolbox IK:")
print(f"    = {np.degrees(sol.q[0]):.2f}°")
print(f"    = {np.degrees(sol.q[1]):.2f}°")
print(f"    Success: {sol.success}")

# Compare with your implementation
theta1, theta2 = ik_2link(1.0, 0.5, L1, L2, 'up')
print(f"\nYour IK (elbow up):")
print(f"    = {np.degrees(theta1):.2f}°")
print(f"    = {np.degrees(theta2):.2f}°")
```

9.2 8.2 Explore Different Initial Guesses

```
# Different initial guesses yield different solutions
print("Effect of initial guess on numerical IK:")
print("-" * 50)

initial_guesses = [
    [0, 0],
    [0, np.pi/2],
    [0, -np.pi/2],
    [np.pi/2, 0],
]

for q0 in initial_guesses:
    sol = robot.ikine_LM(T_target, q0=q0)
    if sol.success:
        print(f"q0={np.degrees(q0)} ->  ={np.degrees(sol.q[0]):7.2f}°,  ={np.degrees(sol.q[1]):7.2f}°")
```

10 Summary and Cleanup

10.1 Checklist

Before leaving, verify you have:

- ☐ Working `is_reachable()` function
- ☐ Working `ik_2link()` function with both elbow configurations
- ☐ Verified all IK solutions with FK (error < 1e-6)
- ☐ Visualization of elbow up vs elbow down saved
- ☐ Tested boundary cases and understand singularities
- ☐ Implemented `ik_nearest()` for solution selection
- ☐ Created path-following animation
- ☐ Completed all exercises with recorded answers

10.2 Save Your Work

```
# Save all your code in a single file
# Organize figures in lab3_figures/ directory
import os
os.makedirs('lab3_figures', exist_ok=True)
```

10.3 Key Takeaways

1. **Always check reachability** before attempting IK
2. **Use `atan2`** instead of `atan` for correct quadrant handling
3. **Two solutions exist** for most reachable points (elbow up/down)
4. **Singularities** occur at workspace boundaries
5. **Verify with FK** - $\text{IK}(\text{FK}(\mathbf{q}))$ should return original angles
6. **Solution selection** depends on application requirements

10.4 Next Week Preview

Week 5: Velocity Kinematics & Jacobians

- Relating joint velocities to end-effector velocities
- The manipulator Jacobian
- Singularity analysis using the Jacobian
- Resolved-rate motion control